

Python Master Notes

Part 7 — Interview Mastery & Final Revision

A complete revision and interview workbook: glossary, core interview concepts, tricky output questions, 80 coding problems with solutions, twelve cheat sheets, and a final mastery checklist.

- Terminology glossary (31 essential terms)
- 24 must-know interview concepts, explained with code
- 44 tricky output questions across 5 difficulty levels
- 30 beginner + 30 intermediate + 20 advanced coding problems
- 12 final cheat sheets
- Final Python -> AI/ML mastery checklist

Companion to Parts 1-6 of the Python Master Notes series.

Table of Contents

SECTION 1 - Terminology Glossary

SECTION 2 - Most Common Interview Concepts

SECTION 3 - Tricky Output Questions

SECTION 4 - Coding Interview: 30 Beginner Problems

SECTION 5 - Coding Interview: 30 Intermediate Problems

SECTION 6 - Coding Interview: 20 Advanced Problems

SECTION 7 - Final Cheat Sheets

- 7.1 Python syntax cheat sheet
- 7.2 Built-in functions cheat sheet
- 7.3 String methods cheat sheet
- 7.4 List methods cheat sheet
- 7.5 Set methods cheat sheet
- 7.6 Dictionary methods cheat sheet
- 7.7 Collections module cheat sheet
- 7.8 DSA templates
- 7.9 Big-O cheat sheet
- 7.10 Python interview traps
- 7.11 Most important terminology (quick list)
- 7.12 One-page final revision sheet

SECTION 8 - Final Mastery Checklist

SECTION 1 — Terminology Glossary

Every term below is one you can be asked to define cold in an interview. Read once, then use it as a lookup while working through the rest of this book.

mutable

An object whose contents can change after creation, in place, keeping the same identity.

list, dict, set, bytearray. `a=[1]; a.append(2)` (same object)

immutable

An object that can never change after creation. 'Changing' it really builds a new object.

int, float, str, tuple, bool, frozenset, bytes. `s='hi'; s+='!'` makes a new str

iterable

Anything you can loop over. It has `__iter__()` (or `__getitem__`) and can hand out an iterator.

list, str, tuple, dict, set, range, file, generator

iterator

An object that produces values one at a time. Has `__iter__()` returning itself and `__next__()`. Raises `StopIteration` when finished. Single-use.

`it=iter([1,2]); next(it)` gives 1

generator

A special iterator created by a function containing `yield` (or by a generator expression). Produces values lazily and remembers its state.

`def f():\n yield 1; (x*x for x in range(5))`

decorator

A function that takes a function and returns a (usually wrapped) function, adding behaviour without editing the original. `@dec` is sugar for `f = dec(f)`.

`@timer` above `def work():`

closure

An inner function that remembers variables of the enclosing function even after the outer function has returned.

`def mul(n):\n def f(x): return x*n\n return f`

namespace

A mapping from names to objects. Python has several: local, enclosing, global, built-in, plus each module/class/instance has its own.

`globals(), locals(), vars(obj)`

scope

The region of code where a name is visible. Decided by where the name is assigned.

A variable assigned inside a function is local to it

LEGB

Lookup order for a name: Local, Enclosing, Global, Built-in. First match wins.

len is found in Built-in unless you shadow it

duck typing

'If it walks like a duck and quacks like a duck...' Python cares about what an object can DO (its methods), not its declared class.

Anything with `__len__` works with `len()`

dynamic typing

Types belong to objects, not variables. A name can be rebound to any type at runtime; type checks happen when code runs.

`x=5; x='five'` is legal

strong typing

Python will not silently coerce unrelated types. Mixing them raises `TypeError` instead of guessing.

`'5'+5` raises `TypeError`

hashable

An object with a hash value that never changes during its life (and `__eq__`). Required for dict keys and set members.

int, str, tuple of hashables. NOT list, dict, set

hash table

Structure that maps `hash(key)` to a slot, giving average $O(1)$ insert/lookup/delete. Powers dict and set.

`d[k], x in s`

object identity

Which exact object something is (its memory identity), independent of its value. Compared with `is`, seen with `id()`.

`[1] == [1]` True, `[1] is [1]` False

reference

A name pointing to an object. Assignment copies the reference, never the object.

`b = a` makes two names for one object

shallow copy

New outer container, but the inner objects are shared with the original.

`a.copy()`, `a[:]`, `list(a)`, `copy.copy(a)`

deep copy

New outer container AND recursively new copies of everything inside.

`copy.deepcopy(a)`

recursion

A function that calls itself on a smaller problem. Needs a base case and progress toward it. Depth limit about 1000 by default.

```
def fact(n): return 1 if n<2 else n*fact(n-1)
```

lambda

A small anonymous function limited to a single expression. Returns that expression's value automatically.

```
sorted(w, key=lambda s: len(s))
```

comprehension

Compact expression that builds a list, set, dict (or generator) from an iterable with optional filter.

```
[x*x for x in a if x%2==0]
```

context manager

An object that sets something up and guarantees cleanup, used with with. Implements `__enter__` and `__exit__`.

```
with open('f.txt') as f:
```

module

A single .py file (or built-in/extension) that you can import. It is one namespace.

```
import math
```

package

A directory of modules (traditionally with `__init__.py`) that you can import as a unit.

```
import os.path, from collections import deque
```

exception

An object signalling an error/event that interrupts normal flow until it is caught by except.

```
ZeroDivisionError, KeyError, custom class MyErr(Exception)
```

polymorphism

One interface, many behaviours. The same method call does different things depending on the object.

```
for s in shapes: s.area()
```

encapsulation

Bundling data with the methods that use it and controlling outside access. In Python by convention: `__protected`, `__private` (name-mangled), properties.

```
self._balance, @property
```

abstraction

Exposing what an object does, hiding how. Often enforced with abstract base classes.

```
abc.ABC and @abstractmethod
```

inheritance

A class reusing and extending another class's attributes and methods.

```
class Dog(Animal):
```

MRO

Method Resolution Order: the order Python searches classes for an attribute, computed by C3 linearization. Seen via `Class.__mro__`.

```
D.__mro__ for diamond inheritance
```

SECTION 2 — Most Common Interview Concepts

These are the concepts that come up again and again in Python interviews. For each one: what it means, working code, and the one-line takeaway to remember.

1. is vs ==

== asks: do the two objects have equal VALUE? is asks: are they the SAME object? Use is only for None/True/False (singletons).

```
a = [1, 2]
b = [1, 2]
c = a
print(a == b, a is b, a is c) # True False True
x = None
print(x is None) # True (preferred over x == None)
```

Takeaway: Value equality -> ==. Same object -> is. Small-int and string caching makes 'is' unreliable for numbers/strings.

2. mutable vs immutable

Mutable objects change in place (list, dict, set). Immutable ones cannot (int, str, tuple). Only immutables (of hashables) can be dict keys.

```
s = "hi"
t = s
s += "!" # builds a NEW str
print(s, t) # hi! hi
a = [1]
b = a
a += [2] # mutates in place
print(a, b) # [1, 2] [1, 2]
```

Takeaway: Rebinding a name never affects other names. Mutating an object affects every name pointing to it.

3. list copy traps

b = a copies nothing. [[0]*3]*3 repeats ONE inner list three times. Default copies are shallow.

```
a = [1, 2, 3]
b = a
b.append(4)
print(a) # [1, 2, 3, 4]
grid = [[0]*3]*3
grid[0][0] = 9
print(grid) # [[9,0,0],[9,0,0],[9,0,0]]
ok = [[0]*3 for _ in range(3)]
ok[0][0] = 9
print(ok) # [[9,0,0],[0,0,0],[0,0,0]]
```

Takeaway: Build 2D lists with a comprehension. Copy on purpose with .copy() / deepcopy.

4. shallow vs deep copy

Shallow copy: new outer list, shared inner objects. Deep copy: everything duplicated recursively.

```
import copy
a = [[1, 2], [3, 4]]
sh = a.copy()
dp = copy.deepcopy(a)
a[0].append(99)
print(sh) # [[1, 2, 99], [3, 4]] shared inner list
print(dp) # [[1, 2], [3, 4]] independent
```

Takeaway: Shallow is enough for flat lists of immutables. Nested mutables need deepcopy.

5. append() vs extend()

append(x) adds x as ONE element. extend(it) adds each element of an iterable. += on a list behaves like extend.

```
a = [1, 2]
a.append([3, 4]); print(a) # [1, 2, [3, 4]]
b = [1, 2]
b.extend([3, 4]); print(b) # [1, 2, 3, 4]
c = [1]
c.extend("ab"); print(c) # [1, 'a', 'b']
```

Takeaway: Both return None and modify in place. append is O(1), extend is O(k).

6. sort() vs sorted()

list.sort() sorts in place and returns None (lists only). sorted() returns a NEW list from any iterable. Both are stable, Timsort, O(n log n).

```
a = [3, 1, 2]
print(sorted(a), a) # [1, 2, 3] [3, 1, 2]
print(a.sort(), a) # None [1, 2, 3]
words = ["bb", "a", "ccc"]
print(sorted(words, key=len, reverse=True)) # ['ccc', 'bb', 'a']
```

Takeaway: Never write a = a.sort(); you get None.

7. remove() vs pop() vs del

remove(v) deletes the first item equal to v (ValueError if absent). pop(i) deletes by index and RETURNS it (default last). del a[i] / del a[i:j] deletes by index/slice, returns nothing.

```
a = [10, 20, 30, 20]
a.remove(20); print(a) # [10, 30, 20]
x = a.pop(); print(x, a) # 20 [10, 30]
del a[0]; print(a) # [30]
b = list(range(6))
del b[1:4]; print(b) # [0, 4, 5]
```

Takeaway: remove = by value, pop = by index + returns, del = by index/slice or whole name. pop(0) and remove are O(n).

8. list vs tuple

List: mutable, variable length, unhashable. Tuple: immutable, hashable if contents are, slightly faster/smaller. Use tuples for fixed records and dict keys.

```
pt = (3, 4)
d = {pt: "home"} # OK, tuple is hashable
# d[[3,4]] = 1 # TypeError: unhashable
single = (5,) # comma makes the tuple
print(type((5)), type(single)) # int tuple
```

Takeaway: A tuple is immutable but a tuple containing a list can still have that list mutated.

9. set vs dictionary

Set: unique elements, no values, fast membership. Dict: unique keys mapped to values. Both are hash tables with average $O(1)$ lookup. `{}` is an empty DICT; use `set()` for an empty set.

```
s = {1, 2, 2, 3} # {1, 2, 3}
d = {"a": 1, "b": 2}
print(type({}), type(set())) # dict set
print(2 in s, "a" in d) # True True (d checks KEYS)
```

Takeaway: Need uniqueness/membership -> set. Need a lookup/count/mapping -> dict.

10. generator vs list

A list stores every element in memory now. A generator computes one element at a time on demand, and can be used once.

```
import sys
lst = [x for x in range(1000000)]
gen = (x for x in range(1000000))
print(sys.getsizeof(lst) > sys.getsizeof(gen)) # True
print(sum(gen), sum(gen)) # 4999950000 0 (exhausted!)
```

Takeaway: Use generators for big/streaming data or one pass. Use lists when you need indexing, len, or repeat passes.

11. iterable vs iterator

An iterable can produce an iterator (`iter(x)`). An iterator produces values with `next()` and remembers position. Every iterator is iterable; not every iterable is an iterator.

```
nums = [1, 2, 3] # iterable
it = iter(nums) # iterator
print(next(it), next(it)) # 1 2
print(list(it)) # [3] (continues where it stopped)
print(list(it)) # [] (exhausted)
```

Takeaway: for-loops call `iter()` once then `next()` until `StopIteration`.

12. return vs yield

return ends the function and hands back one value. yield pauses the function, hands back a value, and resumes later with local state intact. A function with yield returns a generator object.

```
def up_to(n):
    for i in range(n):
        yield i
g = up_to(3)
print(next(g), next(g), next(g)) # 0 1 2
# next(g) now raises StopIteration
def r():
    return 1
    print("never runs")
```

Takeaway: yield = lazy sequence. return = single result and exit.

13. *args vs **kwargs

*args collects extra positional arguments into a tuple. **kwargs collects extra keyword arguments into a dict. In a CALL, * and ** unpack.

```
def f(a, *args, **kwargs):
    print(a, args, kwargs)
f(1, 2, 3, x=4, y=5) # 1 (2, 3) {'x': 4, 'y': 5}
nums = [1, 2]; opts = {"x": 9}
f(*nums, **opts) # 1 (2,) {'x': 9}
```

Takeaway: Order in a def: normal, *args, keyword-only, **kwargs.

14. map() vs filter()

map(f, it) TRANSFORMS every element. filter(f, it) KEEPS elements where f is truthy. Both return lazy iterators in Python 3.

```
a = [1, 2, 3, 4]
print(list(map(lambda x: x*x, a))) # [1, 4, 9, 16]
print(list(filter(lambda x: x % 2 == 0, a))) # [2, 4]
print(list(map(int, "1 2 3".split()))) # [1, 2, 3]
```

Takeaway: Comprehensions are usually more readable: [x*x for x in a], [x for x in a if x%2==0].

15. any() vs all()

any is True if at least one element is truthy (OR). all is True only if every element is truthy (AND). Both short-circuit. any([]) is False, all([]) is True.

```
a = [2, 4, 5]
print(any(x % 2 for x in a)) # True (5 is odd)
print(all(x % 2 == 0 for x in a)) # False
print(any([]), all([])) # False True
```

Takeaway: Empty input: any -> False, all -> True (vacuous truth).

16. range()

`range(start, stop, step)` is a lazy sequence; stop is EXCLUDED. It is an immutable, indexable object with $O(1)$ len and membership for ints.

```
print(list(range(5))) # [0,1,2,3,4]
print(list(range(2, 10, 3))) # [2, 5, 8]
print(list(range(5, 0, -1))) # [5,4,3,2,1]
print(range(10)[3], len(range(0, 10, 2))) # 3 5
```

Takeaway: Reverse loop: `range(n-1, -1, -1)`. `range(n)` is $0..n-1$.

17. lambda

An anonymous single-expression function. Best as a throwaway key= or callback. Not for multi-statement logic.

```
pairs = [(1, 'b'), (2, 'a'), (3, 'c')]
print(sorted(pairs, key=lambda p: p[1]))
# [(2,'a'), (1,'b'), (3,'c')]
add = lambda a, b: a + b
print(add(2, 3)) # 5
```

Takeaway: Late-binding trap: lambdas in a loop capture the variable, not its value at that time.

18. decorators

A decorator wraps a function. Use `functools.wraps` to keep the original name/docstring. `@d` above `def` `f` equals `f = d(f)`.

```
import functools
def logged(fn):
    @functools.wraps(fn)
    def wrapper(*a, **k):
        print("calling", fn.__name__)
        return fn(*a, **k)
    return wrapper

@logged
def add(x, y): return x + y
print(add(2, 3)) # prints 'calling add' then 5
```

Takeaway: Pattern: `outer(fn) -> wrapper(*a, **k) -> call fn -> return result`.

19. closures

The inner function keeps access to the enclosing variables. To REASSIGN an enclosing variable use `nonlocal`.

```
def counter():
    n = 0
    def inc():
        nonlocal n
        n += 1
        return n
    return inc
c = counter()
print(c(), c(), c()) # 1 2 3
```

Takeaway: Without `nonlocal`, `n += 1` makes `n` local and raises `UnboundLocalError`.

20. scope

Names assigned in a function are local. Reading a global is fine; assigning needs global (or nonlocal for enclosing). LEGB decides lookup.

```
x = 10
def show():
    print(x) # reads global: 10
def bad():
    # print(x) # UnboundLocalError if x assigned below
    x = 20
def change():
    global x
    x = 99
change(); print(x) # 99
```

Takeaway: If a name is assigned anywhere in a function, it is local for the WHOLE function.

21. default argument traps

Default values are evaluated ONCE, when def runs. A mutable default is shared across calls.

```
def add(x, bag=[]):
    bag.append(x)
    return bag
print(add(1), add(2)) # [1, 2] [1, 2] (same list!)

def add_ok(x, bag=None):
    if bag is None:
        bag = []
    bag.append(x)
    return bag
print(add_ok(1), add_ok(2)) # [1] [2]
```

Takeaway: Use None as the default and create the mutable object inside.

22. exception handling

try runs code. except handles matching errors (most specific first). else runs if NO error. finally ALWAYS runs. raise re-throws or creates errors.

```
def parse(s):
    try:
        n = int(s)
    except ValueError:
        print("bad input"); return None
    else:
        print("ok"); return n
    finally:
        print("done")
parse("12") # ok, done
parse("x") # bad input, done
```

Takeaway: Never use a bare except:. Catch specific exceptions. finally runs even after return.

23. OOP

Class = blueprint, object = instance. `self` is the instance passed automatically. Instance variables live on `self`, class variables are shared. Dunder methods customise behaviour.

```
class Account:
    bank = "PyBank" # class variable
    def __init__(self, owner, bal=0):
        self.owner = owner # instance variable
        self.bal = bal
    def __str__(self):
        return f"{self.owner}: {self.bal}"
    def deposit(self, amt):
        self.bal += amt
        return self
a = Account("Asha").deposit(50)
print(a, Account.bank) # Asha: 50 PyBank
```

Takeaway: Four pillars: encapsulation, abstraction, inheritance, polymorphism.

24. MRO

With multiple inheritance Python searches classes in C3 linearization order: children before parents, left-to-right, each class once. `super()` follows the MRO, not just 'the parent'.

```
class A:
    def hi(self): return "A"
class B(A):
    def hi(self): return "B>" + super().hi()
class C(A):
    def hi(self): return "C>" + super().hi()
class D(B, C):
    def hi(self): return "D>" + super().hi()
print(D().hi()) # D>B>C>A
print([k.__name__ for k in D.__mro__])# ['D', 'B', 'C', 'A', 'object']
```

Takeaway: Diamond problem solved by MRO. Inspect with `Class.__mro__` or `Class.mro()`.

SECTION 3 — Tricky Output Questions

Predict the output before reading the answer. These progress from warm-up to interview-killer difficulty, covering references, mutation, scope, generators, and OOP.

LEVEL 1 - Warm-up

01. [references]

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

Why: `b = a` copies the reference, not the list. Both names point to one list object, so the append through `b` is visible through `a`.

02. [mutation]

```
a = [1, 2, 3]
print(a.append(4))
print(a)
```

Why: `append` mutates in place and returns `None`. The first print shows that `None`; the second shows the mutated list.

03. [slicing]

```
s = "abcdef"
print(s[1:4], s[::-1], s[-2:], s[::2])
```

Why: Slices are `[start:stop:step]`, stop excluded. `[1:4]` = b,c,d. Step `-1` reverses. `[-2:]` = last two chars. Step `2` takes every second char.

04. [loops]

```
for i in range(3):
    pass
print(i)
```

Why: A `for` loop does not create its own scope. After the loop `i` still holds the last value, `2`.

05. [functions]

```
def f(x):
    x = x + 1
    return x
n = 5
f(n)
print(n)
```

Why: Ints are immutable and `x = x + 1` rebinds the local name `x` to a new object. The caller's `n` is untouched. The returned value was simply discarded.

06. [comprehensions]

```
print([x*x for x in range(6) if x % 2])
```

Why: The `if x % 2` filter keeps odd numbers (1,3,5) and the expression squares them.

07. [exceptions]

```
try:
    print(1 / 0)
except ZeroDivisionError:
    print("zero")
finally:
    print("done")
```

Why: The division raises before print runs. The except block handles it, then finally runs regardless.

LEVEL 2 - Intermediate

08. [slicing]

```
a = [0, 1, 2, 3, 4, 5]
b = a[:]
b[0] = 99
c = a[1:3]
c[0] = 77
print(a, b, c)
```

Why: A slice makes a NEW list (shallow copy). Changing b or c never touches a, because the elements are ints (immutable, rebinding only).

09. [references]

```
x = [1, 2]
y = [x, x]
y[0].append(3)
print(y)
```

Why: y holds two references to the same list x. Appending through y[0] changes that one list, and it shows up in both positions.

010. [mutation]

```
a = (1, 2, [3, 4])
a[2].append(5)
print(a)
```

Why: A tuple is immutable, meaning its references cannot change. But the list inside is a mutable object, so it can still be modified in place.

011. [functions]

```
def add_item(item, lst=[]):
    lst.append(item)
    return lst
print(add_item(1))
print(add_item(2))
print(add_item(3, []))
```

Why: The default list is created ONCE at definition time and shared by calls that omit lst. The third call passes its own fresh list, so it is independent.

O12. [loops]

```
nums = [1, 2, 3, 4]
for n in nums:
    if n % 2 == 0:
        nums.remove(n)
print(nums)
```

Why: Removing while iterating shifts the indexes. Trace: index 0 sees 1; index 1 sees 2 and removes it, so the list becomes [1,3,4]; index 2 now sees 4 and removes it. The element 3 was skipped entirely. Never mutate a list while looping over it; loop over a copy or build a new list.

O13. [scope]

```
x = 5
def show():
    print(x)
def change():
    x = 10
change()
show()
print(x)
```

Why: change assigns x, so x is a LOCAL variable there and the global is never touched. show only reads x, found in the global scope.

O14. [comprehensions]

```
pairs = [(i, j) for i in range(3) for j in range(i)]
print(pairs)
```

Why: Nested loops read left to right like nested for-loops. For i=0, range(0) is empty. For i=1, j=0. For i=2, j=0,1.

O15. [generators]

```
g = (x * 2 for x in range(3))
print(list(g))
print(list(g))
```

Why: A generator is exhausted after one pass. The second list(g) finds nothing left, giving an empty list.

O16. [exceptions]

```
def f():
    try:
        return "try"
    finally:
        print("finally")
print(f())
```

Why: finally always runs before the function actually returns, so 'finally' prints first, then the returned value is printed.

LEVEL 3 - Tricky

O17. [scope]

```
x = 1
def f():
    print(x)
    x = 2
f()
```

Why: Because `x = 2` appears in `f`, `x` is local for the ENTIRE function. Reading it before assignment raises `UnboundLocalError`, not the global value 1.

O18. [loops]

```
funcs = []
for i in range(3):
    funcs.append(lambda: i)
print([f() for f in funcs])
```

Why: Late binding: each lambda looks up `i` when CALLED, not when defined. After the loop, `i` is 2, so all lambdas return 2. Fix: `lambda i=i: i`.

O19. [references]

```
a = [1, 2, 3]
b = a
a = a + [4]
print(a, b)
```

Why: `a + [4]` creates a NEW list and rebinds `a`. `b` still points to the original list, so it is unchanged.

O20. [mutation]

```
a = [1, 2, 3]
b = a
a += [4]
print(a, b)
```

Why: `+=` on a list calls `__iadd__`, which extends IN PLACE and keeps the same object. Compare with `a = a + [4]`, which rebinds. Here both names see the change.

O21. [slicing]

```
a = [1, 2, 3, 4, 5]
a[1:3] = [9]
print(a)
a[::2] = ["x", "y", "z"]
print(a)
```

Why: A plain slice assignment may change the length: `[2,3]` replaced by `[9]` gives `[1,9,4,5]`. An EXTENDED slice (one with a step) must be replaced by a sequence of exactly the same size. `a[::2]` selects 2 positions of the 4-item list, but 3 values are supplied, so `ValueError` is raised.

O22. [default arguments]

```
def f(a, b=[]):
    b.append(a)
    return b
x = f(1)
y = f(2)
print(x is y, x)
```

Why: Both calls return the SAME default list object, so `x is y` is `True` and it contains both appended values.

023. [comprehensions]

```
x = 10
r = [x for x in range(3)]
print(x, r)
```

Why: In Python 3 a list comprehension has its own scope, so its x does not leak. The outer x stays 10.

024. [generators]

```
def gen():
    yield 1
    yield 2
    return 3
g = gen()
print(next(g), next(g))
try:
    next(g)
except StopIteration as e:
    print("value:", e.value)
```

Why: return 3 in a generator ends it by raising StopIteration whose .value is 3. The value is not yielded.

025. [iterators]

```
it = iter([1, 2, 3])
print(list(it))
print(next(it, "empty"))
```

Why: list(it) consumes the iterator. Afterwards next with a default returns that default instead of raising StopIteration.

026. [OOP]

```
class A:
    items = []
    def add(self, x):
        self.items.append(x)
a1, a2 = A(), A()
a1.add(1)
a2.add(2)
print(a1.items, A.items)
```

Why: items is a CLASS variable shared by all instances. self.items.append mutates that shared list, it does not create an instance attribute.

027. [exceptions]

```
def f():
    for i in range(3):
        try:
            if i == 1:
                continue
            print("body", i)
        finally:
            print("cleanup", i)
f()
```

Why: finally runs on every iteration, even when continue skips the rest of the body. For i=1 only the cleanup line prints.

LEVEL 4 - Very tricky

028. [references]

```
a = [[]] * 3
a[0].append(1)
print(a)
b = [[] for _ in range(3)]
b[0].append(1)
print(b)
```

Why: `[[]] * 3` repeats one reference to the same inner list three times. The comprehension creates three separate lists.

029. [mutation]

```
a = [1, 2, 3]
for x in a:
    a.append(x)
    if len(a) > 6:
        break
print(a)
```

Why: Appending while iterating extends the list under the iterator, so the loop keeps finding new items. The break at length 7 prevents an infinite loop.

030. [functions]

```
def f(x, y=[]):
    y = y + [x]
    return y
print(f(1), f(2))
```

Why: `y = y + [x]` creates a NEW list and rebinds the local `y`, so the shared default is never mutated. The trap is avoided by accident.

031. [scope]

```
def outer():
    n = 0
    def inner():
        n += 1
        return n
    return inner
print(outer())()
```

Why: Assignment via `+=` makes `n` local to `inner`, but it has no value yet: `UnboundLocalError`. `nonlocal n` would fix it.

032. [comprehensions]

```
matrix = [[1, 2], [3, 4], [5, 6]]
print([row[i] for row in matrix for i in range(2)])
print([[row[i] for row in matrix] for i in range(2)])
```

Why: First: a flat comprehension, outer loop over rows, inner over columns. Second: nested comprehension that builds a transpose (list of columns).

033. [generators]

```
def g():
    x = yield 1
    print("got", x)
    y = yield 2
    print("got", y)
gen = g()
print(next(gen))
print(gen.send("A"))
try:
    gen.send("B")
except StopIteration:
    print("end")
```

Why: next runs to the first yield. send('A') resumes, making the yield expression evaluate to 'A' (printed as 'got A') and runs to the next yield, returning 2. send('B') prints 'got B' and the generator finishes.

034. [iterators]

```
a = [1, 2, 3]
it1 = iter(a)
it2 = iter(a)
next(it1)
print(next(it1), next(it2))
z = zip(a, a)
print(list(z), list(z))
```

Why: Each iter() gives an independent iterator over the same list. zip returns a one-shot iterator, so the second list(z) is empty.

035. [OOP]

```
class P:
    def __init__(self):
        self.v = 1
    def show(self):
        print(self.v)
class C(P):
    v = 5
c = C()
c.show()
print(C.v, c.v)
```

Why: __init__ sets an INSTANCE attribute v=1, which shadows the class attribute v=5. c.v finds the instance one first. C.v reads the class attribute.

036. [exceptions]

```
def f():
    try:
        raise ValueError("a")
    except ValueError:
        return "except"
    finally:
        return "finally"
print(f())
```

Why: A return inside finally overrides the earlier return (and would even swallow an exception). Avoid it.

LEVEL 5 - Interview killers

037. [references]

```
a = int("5"); b = int("5")
c = int("1000"); d = int("1000")
print(a is b, c is d, c == d)
```

Why: CPython caches small ints (-5..256), so both `int('5')` calls give the SAME object. `int('1000')` builds two separate objects at runtime, so `is` is False while `==` is True. Never use `is` to compare numbers.

038. [functions]

```
def f(*args, **kwargs):
    print(args, kwargs)
f(1, 2, a=3)
f(*[1, 2], **{"a": 3})
f([1, 2], {"a": 3})
```

Why: Calls 1 and 2 are equivalent: `*` and `**` unpack at the call site. Call 3 passes the list and dict as two ordinary positional arguments.

039. [scope]

```
x = "global"
class K:
    x = "class"
    def m(self):
        return x
    ys = [x for _ in range(2)]
print(K().m(), K.ys)
```

Why: Methods do not see class-level names (class scope is skipped by LEGB), so `m` finds the global `x`. In this comprehension the outermost iterable is evaluated in class scope, but the element expression `x` runs in the comprehension's own scope and skips the class, so it sees the global too.

040. [generators]

```
def gen():
    for i in range(3):
        yield i
g = gen()
print(2 in g, 0 in g, list(g))
```

Why: `2 in g` consumes values 0,1,2 until it finds 2. Then `0 in g` continues from where `g` stopped, finds nothing and exhausts it. The remaining list is empty.

041. [OOP]

```
class A:
    def __init__(self): print("A")
class B(A):
    def __init__(self): print("B"); super().__init__()
class C(A):
    def __init__(self): print("C"); super().__init__()
class D(B, C):
    def __init__(self): print("D"); super().__init__()
D()
```

Why: `super()` follows the MRO (D, B, C, A), not just the direct parent. So B's super call goes to C, not A. Each `__init__` runs exactly once.

042. [00P1]

```
class Meta:
    def __eq__(self, other): return True
a, b = Meta(), Meta()
print(a == b, a != b, a is b)
try:
    print({a: 1})
except TypeError as e:
    print("TypeError")
```

Why: Defining `__eq__` without `__hash__` sets `__hash__` to `None`, making instances unhashable, so using one as a dict key raises `TypeError`. `!=` derives from `==` by default (negating it).

043. [exceptions1]

```
def f():
    try:
        raise KeyError("k")
    except KeyError as e:
        raise ValueError("v") from e
try:
    f()
except ValueError as e:
    print(type(e.__cause__).__name__, e)
```

Why: `raise ... from e` chains exceptions and stores the original in `__cause__`. Printing the `ValueError` shows its own message, `v`.

044. [mutation1]

```
a = [1, 2, 3]
a[len(a):] = [4, 5]
a[:0] = [0]
b = a
a = a[::-1]
print(a, b)
```

Why: Assigning to `a[len(a):]` appends, `a[:0]` prepends, both in place. `a[::-1]` builds a new reversed list and rebinds `a`, leaving `b` as the unreversed original.

SECTION 4 — Coding Interview: 30 Beginner Problems

Foundation problems covering strings, arrays, math, sets, and hash maps. Solve these until they take no real thought.

1. Reverse a String [strings]

Approach: Slice with step -1: a string is a sequence and slicing builds a reversed copy.

```
def reverse(s):  
    return s[::-1]
```

Complexity: O(n) time, O(n) space

2. Palindrome Check [strings / two pointers]

Approach: Compare the cleaned string with its reverse. Keep only alphanumerics and lower-case them.

```
def is_palindrome(s):  
    t = [c.lower() for c in s if c.isalnum()]  
    return t == t[::-1]
```

Complexity: O(n) time, O(n) space

3. Count Vowels [strings]

Approach: Loop once. test membership in a string of vowels.

```
def count_vowels(s):  
    return sum(1 for c in s.lower() if c in "aeiou")
```

Complexity: O(n) time, O(1) space

4. FizzBuzz [loops / modulo]

Approach: Test 15 first (both). then 3. then 5. Build a list of strings.

```
def fizzbuzz(n):  
    out = []  
    for i in range(1, n + 1):  
        if i % 15 == 0: out.append("FizzBuzz")  
        elif i % 3 == 0: out.append("Fizz")  
        elif i % 5 == 0: out.append("Buzz")  
        else: out.append(str(i))  
    return out
```

Complexity: O(n) time

5. Factorial [loops / recursion]

Approach: Multiply 2..n. Iterative avoids recursion depth limits.

```
def factorial(n):  
    r = 1  
    for i in range(2, n + 1):  
        r *= i  
    return r
```

Complexity: O(n) time, O(1) space

6. Nth Fibonacci [loops]

Approach: Keep only the last two values and roll them forward.

```
def fib(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

Complexity: $O(n)$ time, $O(1)$ space

7. Prime Check [math]

Approach: Only test divisors up to $\sqrt{n}$. Handle $n < 2$.

```
def is_prime(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True
```

Complexity: $O(\sqrt{n})$ time

8. Sum of Digits [math / modulo]

Approach: Peel digits with $n \% 10$ and $n // 10$.

```
def digit_sum(n):
    n = abs(n)
    total = 0
    while n:
        total += n % 10
        n //= 10
    return total
```

Complexity: $O(\log n)$ time

9. Max Without max() [arrays]

Approach: Track the best value seen so far.

```
def my_max(a):
    best = a[0]
    for x in a[1:]:
        if x > best:
            best = x
    return best
```

Complexity: $O(n)$ time, $O(1)$ space

10. Second Largest [arrays]

Approach: Track the top two distinct values in one pass.

```
def second_largest(a):
    first = second = float("-inf")
    for x in a:
        if x > first:
            first, second = x, first
        elif first > x > second:
            second = x
    return second if second != float("-inf") else None
```

Complexity: $O(n)$ time, $O(1)$ space

11. Remove Duplicates (keep order) [sets]

Approach: A seen-set gives $O(1)$ membership: append only unseen items.

```
def dedupe(a):
    seen, out = set(), []
    for x in a:
        if x not in seen:
            seen.add(x)
            out.append(x)
    return out
```

Complexity: $O(n)$ time, $O(n)$ space

12. Word Frequency [hash map]

Approach: Lower-case. split. count with a dict and dict.get.

```
def word_freq(text):
    freq = {}
    for w in text.lower().split():
        freq[w] = freq.get(w, 0) + 1
    return freq
```

Complexity: $O(n)$ time, $O(k)$ space

13. Valid Anagram [hash map]

Approach: Two strings are anagrams if their character counts match.

```
from collections import Counter
def is_anagram(a, b):
    return Counter(a) == Counter(b)
```

Complexity: $O(n)$ time

14. Reverse Words in a Sentence [strings]

Approach: split() also collapses extra spaces: reverse the list: join.

```
def reverse_words(s):
    return " ".join(s.split()[::-1])
```

Complexity: $O(n)$ time

15. First Non-Repeating Character [hash map]

Approach: Count once. then scan in original order for count 1.

```
from collections import Counter
def first_unique(s):
    cnt = Counter(s)
    for i, c in enumerate(s):
        if cnt[c] == 1:
            return i
    return -1
```

Complexity: O(n) time, O(1) space (alphabet)

16. Missing Number (0..n) [math / sets]

Approach: Expected sum $n(n+1)/2$ minus actual sum.

```
def missing_number(a):
    n = len(a)
    return n * (n + 1) // 2 - sum(a)
```

Complexity: O(n) time, O(1) space

17. Two Sum [hash map]

Approach: Store value->index. For each x look for target-x among earlier values.

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return []
```

Complexity: O(n) time, O(n) space

18. Contains Duplicate [sets]

Approach: If a set of the list is smaller. something repeated.

```
def has_duplicate(a):
    return len(set(a)) != len(a)
```

Complexity: O(n) time, O(n) space

19. Move Zeros to End [two pointers]

Approach: Write non-zeros forward with a slow pointer. then fill the tail with zeros.

```
def move_zeros(a):
    w = 0
    for x in a:
        if x != 0:
            a[w] = x
            w += 1
    for i in range(w, len(a)):
        a[i] = 0
    return a
```

Complexity: O(n) time, O(1) space

20. Merge Two Sorted Lists [two pointers]

Approach: Compare heads with two indexes. then append the leftover tail.

```
def merge(a, b):
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out + a[i:] + b[j:]
```

Complexity: $O(n+m)$ time, $O(n+m)$ space

21. Rotate List Right by k [slicing]

Approach: Reduce k modulo n and stitch two slices.

```
def rotate(a, k):
    if not a:
        return a
    k %= len(a)
    return a[-k:] + a[:-k] if k else a[:]
```

Complexity: $O(n)$ time, $O(n)$ space

22. GCD (Euclid) [math]

Approach: $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ until b is 0.

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a
```

Complexity: $O(\log \min(a,b))$ time

23. Power of Two [bit tricks]

Approach: A power of two has exactly one set bit. so $n \& (n-1)$ is 0.

```
def is_power_of_two(n):
    return n > 0 and n & (n - 1) == 0
```

Complexity: $O(1)$ time

24. Decimal to Binary String [math / strings]

Approach: Repeatedly divide by 2 and collect remainders: reverse at the end.

```
def to_binary(n):
    if n == 0:
        return "0"
    bits = []
    while n:
        bits.append(str(n % 2))
        n //= 2
    return "".join(reversed(bits))
```

Complexity: $O(\log n)$ time

25. Armstrong Number [math]

Approach: Sum of each digit raised to the digit count equals the number.

```
def is_armstrong(n):
    digits = str(n)
    p = len(digits)
    return sum(int(d) ** p for d in digits) == n
```

Complexity: $O(d)$ time

26. Longest Word [strings]

Approach: `max()` with `key=len` returns the first longest word.

```
def longest_word(s):
    words = s.split()
    return max(words, key=len) if words else ""
```

Complexity: $O(n)$ time

27. Run-Length Encoding [strings]

Approach: Walk the string, counting a run until the character changes.

```
def rle(s):
    if not s:
        return ""
    out, count = [], 1
    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            count += 1
        else:
            out.append(s[i - 1] + str(count))
            count = 1
    out.append(s[-1] + str(count))
    return "".join(out)
```

Complexity: $O(n)$ time

28. List Intersection [sets]

Approach: Convert to sets and use `&`: sorted for stable output.

```
def intersection(a, b):
    return sorted(set(a) & set(b))
```

Complexity: $O(n+m)$ time

29. Leap Year [conditions]

Approach: Divisible by 4, except centuries unless divisible by 400.

```
def is_leap(y):
    return y % 4 == 0 and (y % 100 != 0 or y % 400 == 0)
```

Complexity: $O(1)$ time

30. Transpose a Matrix [lists / zip]

Approach: `zip(*m)` groups the *i*-th element of every row.

```
def transpose(m):  
    return [list(col) for col in zip(*m)]
```

Complexity: $O(r*c)$ time

SECTION 5 — Coding Interview: 30 Intermediate Problems

Pattern-driven problems: sliding window, two pointers, stacks, heaps, prefix sums, binary search, and an introduction to backtracking and DP.

1. Valid Parentheses [stack]

Approach: Push openers. On a closer, the top of the stack must be its matching opener. Stack must be empty at the end.

```
def valid(s):
    pair = {"(": ")", "[": "]", "{": "}"
    st = []
    for c in s:
        if c in pair:
            if not st or st.pop() != pair[c]:
                return False
        else:
            st.append(c)
    return not st
```

Complexity: $O(n)$ time, $O(n)$ space

2. Group Anagrams [hash map]

Approach: Anagrams share the same sorted-letters key. Group in a defaultdict(list).

```
from collections import defaultdict
def group_anagrams(words):
    g = defaultdict(list)
    for w in words:
        g["".join(sorted(w))].append(w)
    return list(g.values())
```

Complexity: $O(n * k \log k)$ time

3. Longest Substring Without Repeats [sliding window]

Approach: Expand right: if the char was seen inside the window. move left past its last index.

```
def longest_unique(s):
    last, left, best = {}, 0, 0
    for right, c in enumerate(s):
        if c in last and last[c] >= left:
            left = last[c] + 1
        last[c] = right
        best = max(best, right - left + 1)
    return best
```

Complexity: $O(n)$ time, $O(k)$ space

4. Max Sum of Size-k Subarray [sliding window]

Approach: Keep a running window sum: add the new element. subtract the one leaving.

```
def max_sum_k(a, k):
    if k > len(a):
        return None
    cur = sum(a[:k])
    best = cur
    for i in range(k, len(a)):
        cur += a[i] - a[i - k]
        best = max(best, cur)
    return best
```

Complexity: O(n) time, O(1) space

5. Maximum Subarray (Kadane) [dynamic idea]

Approach: At each element either extend the current run or start fresh here.

```
def max_subarray(a):
    cur = best = a[0]
    for x in a[1:]:
        cur = max(x, cur + x)
        best = max(best, cur)
    return best
```

Complexity: O(n) time, O(1) space

6. Product of Array Except Self [prefix / suffix]

Approach: Left products pass. then multiply by a running right product. No division.

```
def product_except_self(a):
    n = len(a)
    out = [1] * n
    left = 1
    for i in range(n):
        out[i] = left
        left *= a[i]
    right = 1
    for i in range(n - 1, -1, -1):
        out[i] *= right
        right *= a[i]
    return out
```

Complexity: O(n) time, O(1) extra space

7. Binary Search [binary search]

Approach: Halve the range each step. Use $l \leq r$ and $mid = l + (r-l)//2$.

```
def binary_search(a, target):
    l, r = 0, len(a) - 1
    while l <= r:
        mid = l + (r - l) // 2
        if a[mid] == target:
            return mid
        if a[mid] < target:
            l = mid + 1
        else:
            r = mid - 1
    return -1
```

Complexity: $O(\log n)$ time, $O(1)$ space

8. First and Last Position [binary search / bisect]

Approach: bisect left gives the first index. bisect right minus 1 gives the last.

```
from bisect import bisect_left, bisect_right
def search_range(a, x):
    lo = bisect_left(a, x)
    if lo == len(a) or a[lo] != x:
        return [-1, -1]
    return [lo, bisect_right(a, x) - 1]
```

Complexity: $O(\log n)$ time

9. Search in Rotated Sorted Array [binary search]

Approach: One half is always sorted. Decide which half holds the target.

```
def search_rotated(a, t):
    l, r = 0, len(a) - 1
    while l <= r:
        m = (l + r) // 2
        if a[m] == t:
            return m
        if a[l] <= a[m]:
            if a[l] <= t < a[m]:
                r = m - 1
            else:
                l = m + 1
        else:
            if a[m] < t <= a[r]:
                l = m + 1
            else:
                r = m - 1
    return -1
```

Complexity: $O(\log n)$ time

10. 3Sum [sorting + two pointers]

Approach: Sort. Fix i. then squeeze l and r. Skip duplicates to avoid repeated triplets.

```
def three_sum(a):
    a.sort()
    res = []
    for i in range(len(a) - 2):
        if i and a[i] == a[i - 1]:
            continue
        l, r = i + 1, len(a) - 1
        while l < r:
            s = a[i] + a[l] + a[r]
            if s < 0: l += 1
            elif s > 0: r -= 1
            else:
                res.append([a[i], a[l], a[r]])
                l += 1
                while l < r and a[l] == a[l - 1]:
                    l += 1
        return res
```

Complexity: $O(n^2)$ time

11. Container With Most Water [two pointers]

Approach: Area is limited by the shorter wall. Move the shorter pointer inward.

```
def max_area(h):
    l, r, best = 0, len(h) - 1, 0
    while l < r:
        best = max(best, min(h[l], h[r]) * (r - l))
        if h[l] < h[r]:
            l += 1
        else:
            r -= 1
    return best
```

Complexity: $O(n)$ time, $O(1)$ space

12. Merge Intervals [sorting]

Approach: Sort by start. Extend the last interval if the next one overlaps. else append.

```
def merge_intervals(iv):
    iv.sort(key=lambda x: x[0])
    out = []
    for s, e in iv:
        if out and s <= out[-1][1]:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out
```

Complexity: $O(n \log n)$ time

13. Top K Frequent Elements [heap / Counter]

Approach: Count. then `heapq.nlargest` by count.

```
import heapq
from collections import Counter
def top_k(nums, k):
    cnt = Counter(nums)
    return heapq.nlargest(k, cnt, key=cnt.get)
```

Complexity: $O(n \log k)$ time

14. Kth Largest Element [heap]

Approach: Keep a min-heap of size k . Its root is the k th largest.

```
import heapq
def kth_largest(a, k):
    h = a[:k]
    heapq.heapify(h)
    for x in a[k:]:
        if x > h[0]:
            heapq.heapreplace(h, x)
    return h[0]
```

Complexity: $O(n \log k)$ time, $O(k)$ space

15. Min Stack [stack design]

Approach: Store (value, min so far) pairs so the minimum is $O(1)$.

```
class MinStack:
    def __init__(self):
        self.st = []
    def push(self, x):
        m = min(x, self.st[-1][1]) if self.st else x
        self.st.append((x, m))
    def pop(self):
        return self.st.pop()[0]
    def top(self):
        return self.st[-1][0]
    def get_min(self):
        return self.st[-1][1]
```

Complexity: $O(1)$ per operation

16. Queue Using Two Stacks [stack / queue design]

Approach: Push to an inbox stack. On pop, if outbox is empty, pour inbox into it (reverses order).

```
class MyQueue:
    def __init__(self):
        self.inbox, self.outbox = [], []
    def push(self, x):
        self.inbox.append(x)
    def pop(self):
        if not self.outbox:
            while self.inbox:
                self.outbox.append(self.inbox.pop())
        return self.outbox.pop()
```

Complexity: Amortized $O(1)$ per operation

17. Daily Temperatures [monotonic stack]

Approach: Keep a stack of indexes with decreasing temperatures. A warmer day resolves every colder index on top.

```
def daily_temps(t):
    res = [0] * len(t)
    st = []
    for i, x in enumerate(t):
        while st and t[st[-1]] < x:
            j = st.pop()
            res[j] = i - j
        st.append(i)
    return res
```

Complexity: O(n) time, O(n) space

18. Next Greater Element [monotonic stack]

Approach: Scan right to left keeping a stack of candidates greater than the current value.

```
def next_greater(a):
    res = [-1] * len(a)
    st = []
    for i in range(len(a) - 1, -1, -1):
        while st and st[-1] <= a[i]:
            st.pop()
        if st:
            res[i] = st[-1]
        st.append(a[i])
    return res
```

Complexity: O(n) time

19. Subarray Sum Equals K [prefix sum + hash map]

Approach: If prefix - k was seen before. those subarrays end here and sum to k.

```
from collections import defaultdict
def subarray_sum(a, k):
    cnt = defaultdict(int)
    cnt[0] = 1
    pref = res = 0
    for x in a:
        pref += x
        res += cnt[pref - k]
        cnt[pref] += 1
    return res
```

Complexity: O(n) time, O(n) space

20. Longest Consecutive Sequence [sets]

Approach: Only start counting from numbers whose predecessor is absent.

```
def longest_consecutive(a):
    s = set(a)
    best = 0
    for x in s:
        if x - 1 not in s:
            y = x
            while y + 1 in s:
                y += 1
            best = max(best, y - x + 1)
    return best
```

Complexity: O(n) time, O(n) space

21. Longest Common Prefix [strings]

Approach: Compare every string with the first. character by character (zip(*strs) does it neatly).

```
def common_prefix(strs):
    out = []
    for chars in zip(*strs):
        if len(set(chars)) != 1:
            break
        out.append(chars[0])
    return "".join(out)
```

Complexity: O(total chars) time

22. Reverse a Linked List [pointers]

Approach: Walk the list. flipping each next pointer to the previous node.

```
class Node:
    def __init__(self, val, nxt=None):
        self.val, self.next = val, nxt

def reverse_list(head):
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next
    return prev
```

Complexity: O(n) time, O(1) space

23. Climbing Stairs [dynamic programming]

Approach: $wavs(n) = wavs(n-1) + wavs(n-2)$. Keep two variables.

```
def climb(n):
    a, b = 1, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b
```

Complexity: O(n) time, O(1) space

24. House Robber [dynamic programming]

Approach: best = max(skip this house. rob it + best two houses back).

```
def rob(a):
    prev = cur = 0
    for x in a:
        prev, cur = cur, max(cur, prev + x)
    return cur
```

Complexity: O(n) time, O(1) space

25. All Subsets [recursion / backtracking]

Approach: For each element choose: include it or not. Snapshot the path at every step.

```
def subsets(nums):
    res = []
    def go(i, path):
        if i == len(nums):
            res.append(path[:])
            return
        go(i + 1, path)
        path.append(nums[i])
        go(i + 1, path)
        path.pop()
    go(0, [])
    return res
```

Complexity: O(2ⁿ * n) time

26. Permutations [recursion / backtracking]

Approach: Build the path by choosing any unused element at each level.

```
def permutations(nums):
    res, used = [], [False] * len(nums)
    def go(path):
        if len(path) == len(nums):
            res.append(path[:])
            return
        for i, x in enumerate(nums):
            if not used[i]:
                used[i] = True
                path.append(x)
                go(path)
                path.pop()
                used[i] = False
    go([])
    return res
```

Complexity: O(n! * n) time

27. Maximum Depth of Binary Tree [recursion]

Approach: $\text{depth} = 1 + \max(\text{depth}(\text{left}), \text{depth}(\text{right}))$: empty tree is 0.

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val, self.left, self.right = val, left, right

def max_depth(root):
    if not root:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

Complexity: $O(n)$ time, $O(h)$ space

28. Sort Colors (0,1,2) [two pointers (Dutch flag)]

Approach: Three pointers: low for 0s. high for 2s. mid scans. Swap accordingly.

```
def sort_colors(a):
    lo, mid, hi = 0, 0, len(a) - 1
    while mid <= hi:
        if a[mid] == 0:
            a[lo], a[mid] = a[mid], a[lo]
            lo += 1; mid += 1
        elif a[mid] == 1:
            mid += 1
        else:
            a[mid], a[hi] = a[hi], a[mid]
            hi -= 1
    return a
```

Complexity: $O(n)$ time, $O(1)$ space

29. Rotate Matrix 90 Degrees [matrix]

Approach: Rotating clockwise = transpose. then reverse each row.

```
def rotate_matrix(m):
    n = len(m)
    for i in range(n):
        for j in range(i + 1, n):
            m[i][j], m[j][i] = m[j][i], m[i][j]
    for row in m:
        row.reverse()
    return m
```

Complexity: $O(n^2)$ time, $O(1)$ space

30. Majority Element (Boyer-Moore) [voting / counting]

Approach: Cancel pairs of different elements. The majority survives as the candidate.

```
def majority(a):
    cand, cnt = None, 0
    for x in a:
        if cnt == 0:
            cand = x
        cnt += 1 if x == cand else -1
    return cand
```

Complexity: $O(n)$ time, $O(1)$ space

SECTION 6 — Coding Interview: 20 Advanced Problems

Harder pattern combinations and classic hard interview questions: multi-structure designs, graph algorithms, and 2D dynamic programming.

1. Minimum Window Substring [sliding window + hash map]

Approach: Expand right until all needed chars are covered, then shrink left while still valid, tracking the smallest window.

```
from collections import Counter
def min_window(s, t):
    need = Counter(t)
    missing = len(t)
    left = 0
    best = (float("inf"), 0, 0)
    for right, c in enumerate(s):
        if need[c] > 0:
            missing -= 1
        need[c] -= 1
        if missing == 0:
            while need[s[left]] < 0:
                need[s[left]] += 1
                left += 1
            if right - left + 1 < best[0]:
                best = (right - left + 1, left, right)
            need[s[left]] += 1
            missing += 1
            left += 1
    return "" if best[0] == float("inf") else s[best[1]:best[2] + 1]
```

Complexity: $O(n + m)$ time

2. Sliding Window Maximum [monotonic deque]

Approach: Deque of indexes with decreasing values. Front is the window max; drop out-of-window indexes.

```
from collections import deque
def max_sliding_window(a, k):
    dq, res = deque(), []
    for i, x in enumerate(a):
        while dq and a[dq[-1]] <= x:
            dq.pop()
        dq.append(i)
        if dq[0] <= i - k:
            dq.popleft()
        if i >= k - 1:
            res.append(a[dq[0]])
    return res
```

Complexity: $O(n)$ time, $O(k)$ space

3. Merge K Sorted Lists [heap]

Approach: Heap holds (value, list id, index) of each list's current head. Pop the smallest, push its successor.

```
import heapq
def merge_k(lists):
    h = [(l[0], i, 0) for i, l in enumerate(lists) if l]
    heapq.heapify(h)
    out = []
    while h:
        v, i, j = heapq.heappop(h)
        out.append(v)
        if j + 1 < len(lists[i]):
            heapq.heappush(h, (lists[i][j + 1], i, j + 1))
    return out
```

Complexity: $O(N \log k)$ time

4. Median from a Data Stream [two heaps]

Approach: Max-heap for the lower half. min-heap for the upper half. sizes balanced within 1.

```
import heapq
class MedianFinder:
    def __init__(self):
        self.lo, self.hi = [], [] # lo is a max-heap (negated)
    def add(self, x):
        heapq.heappush(self.lo, -x)
        heapq.heappush(self.hi, -heapq.heappop(self.lo))
        if len(self.hi) > len(self.lo):
            heapq.heappush(self.lo, -heapq.heappop(self.hi))
    def median(self):
        if len(self.lo) > len(self.hi):
            return -self.lo[0]
        return (-self.lo[0] + self.hi[0]) / 2
```

Complexity: $O(\log n)$ add, $O(1)$ median

5. Trapping Rain Water [two pointers]

Approach: Water above a cell is bounded by the smaller of the max heights on each side. Move the smaller side.

```
def trap(h):
    l, r = 0, len(h) - 1
    lmax = rmax = water = 0
    while l < r:
        if h[l] < h[r]:
            lmax = max(lmax, h[l])
            water += lmax - h[l]
            l += 1
        else:
            rmax = max(rmax, h[r])
            water += rmax - h[r]
            r -= 1
    return water
```

Complexity: $O(n)$ time, $O(1)$ space

6. Largest Rectangle in Histogram [monotonic stack]

Approach: Keep increasing heights on a stack. A shorter bar closes taller ones and fixes their widths.

```
def largest_rectangle(h):
    st, best = [], 0
    for i, x in enumerate(h + [0]):
        start = i
        while st and st[-1][1] >= x:
            idx, height = st.pop()
            best = max(best, height * (i - idx))
            start = idx
        st.append((start, x))
    return best
```

Complexity: $O(n)$ time, $O(n)$ space

7. Coin Change (min coins) [dynamic programming]

Approach: $dp[a] = 1 + \min(dp[a - \text{coin}])$ over all coins. Unreachable stays infinity.

```
def coin_change(coins, amount):
    INF = float("inf")
    dp = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != INF else -1
```

Complexity: $O(\text{amount} * \text{coins})$ time

8. Longest Increasing Subsequence [dp + binary search]

Approach: $\text{tails}[i]$ = smallest tail of an increasing subsequence of length $i+1$. `bisect_left` keeps it sorted.

```
from bisect import bisect_left
def lis(a):
    tails = []
    for x in a:
        i = bisect_left(tails, x)
        if i == len(tails):
            tails.append(x)
        else:
            tails[i] = x
    return len(tails)
```

Complexity: $O(n \log n)$ time

9. Word Break [dynamic programming]

Approach: `dp[i]` is True if `s[:i]` can be segmented: some $i < i$ has `dp[i]` and `s[i:i]` in the word set.

```
def word_break(s, words):
    ws = set(words)
    dp = [True] + [False] * len(s)
    for i in range(1, len(s) + 1):
        for j in range(i):
            if dp[j] and s[j:i] in ws:
                dp[i] = True
                break
    return dp[-1]
```

Complexity: $O(n^2)$ time

10. Number of Islands [DFS on a grid]

Approach: Each unvisited '1' starts a new island. Flood-fill it with an explicit stack.

```
def num_islands(g):
    if not g:
        return 0
    R, C = len(g), len(g[0])
    seen, count = set(), 0
    for r in range(R):
        for c in range(C):
            if g[r][c] == "1" and (r, c) not in seen:
                count += 1
                st = [(r, c)]
                seen.add((r, c))
                while st:
                    x, y = st.pop()
                    for dx, dy in ((1,0), (-1,0), (0,1), (0,-1)):
                        nx, ny = x + dx, y + dy
                        in_bounds = 0 <= nx < R and 0 <= ny < C
                        if in_bounds and g[nx][ny] == "1" and (nx, ny) not in seen:
                            seen.add((nx, ny))
                            st.append((nx, ny))
    return count
```

Complexity: $O(R*C)$ time

11. Course Schedule (cycle detection) [graph / topological sort]

Approach: Kahn's algorithm: repeatedly remove nodes with indegree 0. If all are removed there is no cycle.

```
from collections import deque, defaultdict
def can_finish(n, prereqs):
    adj, indeg = defaultdict(list), [0] * n
    for a, b in prereqs:
        adj[b].append(a)
        indeg[a] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    done = 0
    while q:
        u = q.popleft()
        done += 1
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return done == n
```

Complexity: $O(V + E)$ time

12. Dijkstra Shortest Path [heap / graph]

Approach: Always expand the closest unvisited node using a min-heap of (distance, node).

```
import heapq
def dijkstra(graph, src):
    dist = {src: 0}
    h = [(0, src)]
    while h:
        d, u = heapq.heappop(h)
        if d > dist.get(u, float("inf")):
            continue
        for v, w in graph.get(u, []):
            nd = d + w
            if nd < dist.get(v, float("inf")):
                dist[v] = nd
                heapq.heappush(h, (nd, v))
    return dist
```

Complexity: $O((V + E) \log V)$ time

13. LRU Cache [hash map + ordering]

Approach: OrderedDict keeps usage order. move_to_end on access; popitem(last=False) evicts the oldest.

```
from collections import OrderedDict
class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.d = OrderedDict()
    def get(self, key):
        if key not in self.d:
            return -1
        self.d.move_to_end(key)
        return self.d[key]
    def put(self, key, value):
        self.d[key] = value
        self.d.move_to_end(key)
        if len(self.d) > self.cap:
            self.d.popitem(last=False)
```

Complexity: O(1) per operation

14. N-Queens (count solutions) [backtracking]

Approach: Place one queen per row. Track used columns and both diagonals in sets.

```
def n_queens(n):
    cols, d1, d2 = set(), set(), set()
    def go(r):
        if r == n:
            return 1
        total = 0
        for c in range(n):
            if c in cols or r - c in d1 or r + c in d2:
                continue
            cols.add(c); d1.add(r - c); d2.add(r + c)
            total += go(r + 1)
            cols.remove(c); d1.remove(r - c); d2.remove(r + c)
        return total
    return go(0)
```

Complexity: O(n!) worst case

15. Word Ladder (shortest path) [BFS]

Approach: Each word is a node: neighbours differ by one letter. BFS finds the fewest steps.

```
from collections import deque
def ladder_length(begin, end, words):
    ws = set(words)
    if end not in ws:
        return 0
    q = deque([(begin, 1)])
    seen = {begin}
    while q:
        w, d = q.popleft()
        if w == end:
            return d
        for i in range(len(w)):
            for ch in "abcdefghijklmnopqrstuvwxyz":
                nw = w[:i] + ch + w[i + 1:]
                if nw in ws and nw not in seen:
                    seen.add(nw)
                    q.append((nw, d + 1))
    return 0
```

Complexity: $O(N * L * 26)$ time

16. Edit Distance [dynamic programming]

Approach: $dp[i][j]$ = edits to turn $a[:i]$ into $b[:j]$. Match: copy diagonal. Else $1 + \min(\text{insert, delete, replace})$.

```
def edit_distance(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(m + 1):
        dp[i][0] = i
    for j in range(n + 1):
        dp[0][j] = j
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]
            else:
                dp[i][j] = 1 + min(dp[i - 1][j], dp[i][j - 1], dp[i - 1][j - 1])
    return dp[m][n]
```

Complexity: $O(m*n)$ time and space

17. Longest Palindromic Substring [expand around center]

Approach: Every palindrome has a center (a char or a gap). Expand outward from each of the $2n-1$ centers.

```
def longest_palindrome(s):
    best = ""
    for c in range(len(s)):
        for l, r in ((c, c), (c, c + 1)):
            while l >= 0 and r < len(s) and s[l] == s[r]:
                l -= 1
                r += 1
            if r - l - 1 > len(best):
                best = s[l + 1:r]
    return best
```

Complexity: $O(n^2)$ time, $O(1)$ space

18. Combination Sum [backtracking]

Approach: Choose candidates with unlimited reuse. Keep a start index so combinations are not repeated in different orders.

```
def combination_sum(cands, target):
    res = []
    def go(start, path, remain):
        if remain == 0:
            res.append(path[:])
            return
        for i in range(start, len(cands)):
            if cands[i] <= remain:
                path.append(cands[i])
                go(i, path, remain - cands[i])
                path.pop()
    go(0, [], target)
    return res
```

Complexity: Exponential; bounded by $\text{target}/\min(\text{cands})$ depth

19. Implement a Trie [tree / strings]

Approach: Each node maps a character to a child node. with a flag marking a complete word.

```
class Trie:
    def __init__(self):
        self.root = {}
    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.setdefault(ch, {})
        node["$"] = True
    def _walk(self, s):
        node = self.root
        for ch in s:
            if ch not in node:
                return None
            node = node[ch]
        return node
    def search(self, word):
        n = self._walk(word)
        return bool(n and "$" in n)
    def starts_with(self, prefix):
        return self._walk(prefix) is not None
```

Complexity: $O(L)$ per operation

20. Kth Smallest in Sorted Matrix [heap]

Approach: Rows and columns are sorted. Pop the smallest k times, pushing the next element in that row.

```
import heapq
def kth_smallest(matrix, k):
    n = len(matrix)
    h = [(matrix[r][0], r, 0) for r in range(min(n, k))]
    heapq.heapify(h)
    for _ in range(k - 1):
        v, r, c = heapq.heappop(h)
        if c + 1 < len(matrix[r]):
            heapq.heappush(h, (matrix[r][c + 1], r, c + 1))
    return h[0][0]
```

Complexity: $O(k \log n)$ time

SECTION 7 — Final Cheat Sheets

Twelve compact references. Flip here in the last days before an interview.

7.1 Python Syntax Cheat Sheet

```
# variables, types, conversion
x = 10; name = "Asha"; pi = 3.14; ok = True; nothing = None
int("5"); float("2.5"); str(9); bool([]); list("ab"); tuple([1]); set([1,1]); dict([(1,2)])
a, b = b, a # swap
a, *rest = [1, 2, 3] # unpack -> a=1, rest=[2,3]

# operators
+ - * / // % ** # / true div, // floor div, ** power
== != < > <= >= and or not in not in is is not
& | ^ ~ << >> # bitwise
x if cond else y # ternary
(n := len(a)) # walrus

# control flow
if c1: ...
elif c2: ...
else: ...
for i in range(n): ... # break / continue / else clause
for i, v in enumerate(a): ...
for x, y in zip(a, b): ...
while cond: ...

# functions
def f(a, b=0, *args, key=None, **kw):
    """docstring"""
    return a + b
sq = lambda x: x * x

# comprehensions
[x*x for x in a if x % 2] {x: x*x for x in a} {x for x in a}
(x*x for x in a) # generator expression

# strings / slicing
s[i] s[-1] s[a:b] s[::2] s[::-1] f"{x:.2f} {name!r} {n:>5}"

# exceptions and files
try: ...
except (ValueError, KeyError) as e: ...
else: ...
finally: ...
raise ValueError("msg")
with open("f.txt") as f: data = f.read()

# classes
class Dog(Animal):
    count = 0
    def __init__(self, name):
        super().__init__(name)
```

```

        self.name = name
    def __repr__(self): return f"Dog({self.name!r})"
    @property
    def upper(self): return self.name.upper()
    @staticmethod
    def helper(x): ...
    @classmethod
    def make(cls): return cls("x")

# generators / decorators
def gen(): yield 1
def deco(fn):
    @functools.wraps(fn)
    def w(*a, **k): return fn(*a, **k)
    return w
if __name__ == "__main__": main()

```

7.2 Built-in Functions Cheat Sheet

Function	What it does	Example
<code>len(x)</code>	Number of items	<code>len('abc') -> 3</code>
<code>sum(it, start=0)</code>	Add items	<code>sum([1,2,3]) -> 6</code>
<code>min/max(it, key=)</code>	Smallest / largest	<code>max(words, key=len)</code>
<code>abs(x)</code>	Absolute value	<code>abs(-4) -> 4</code>
<code>round(x, n)</code>	Round (banker's rounding)	<code>round(2.5) -> 2</code>
<code>divmod(a, b)</code>	(quotient, remainder)	<code>divmod(17,5) -> (3,2)</code>
<code>pow(a, b, mod)</code>	Power, optional modulus	<code>pow(2,10,1000) -> 24</code>
<code>sorted(it, key=, reverse=)</code>	New sorted list	<code>sorted('bca') -> ['a','b','c']</code>
<code>reversed(seq)</code>	Reverse iterator	<code>list(reversed([1,2]))</code>
<code>enumerate(it, start=0)</code>	(index, value) pairs	<code>enumerate(a, 1)</code>
<code>zip(*its)</code>	Pair items; stops at shortest	<code>list(zip('ab',[1,2]))</code>
<code>range(a, b, s)</code>	Lazy integer sequence	<code>range(10,0,-2)</code>
<code>map(f, it)</code>	Apply f to each	<code>list(map(int, ['1','2']))</code>
<code>filter(f, it)</code>	Keep truthy f(x)	<code>list(filter(None,[0,1,2]))</code>
<code>any / all</code>	OR / AND of truthiness	<code>all(x>0 for x in a)</code>
<code>iter / next</code>	Get iterator / next value	<code>next(it, default)</code>
<code>type / isinstance</code>	Type check	<code>isinstance(x, (int, float))</code>
<code>id / hash</code>	Identity / hash value	<code>id(a) == id(b)</code>
<code>ord / chr</code>	Char <-> code point	<code>ord('a') -> 97</code>
<code>bin / hex / oct</code>	Number to string base	<code>bin(5) -> '0b101'</code>
<code>int(s, base)</code>	Parse in a base	<code>int('ff', 16) -> 255</code>
<code>input / print</code>	I/O	<code>print(a, b, sep=',', end='')</code>
<code>open(path, mode)</code>	File object	<code>open('f','w')</code>
<code>dir / vars / help</code>	Introspection	<code>dir(list)</code>
<code>getattr / setattr / hasattr</code>	Dynamic attribute access	<code>getattr(o,'x',0)</code>
<code>callable / super</code>	Is callable / parent proxy	<code>callable(len)</code>
<code>list/tuple/set/dict/frozenset</code>	Constructors	<code>set('aab') -> {'a','b'}</code>
<code>repr / str / format</code>	Text forms	<code>repr('a')</code> gives quoted form
<code>slice(a,b,c)</code>	Slice object	<code>a[slice(1,3)]</code>

7.3 String Methods Cheat Sheet

Method	Result / use
s.lower() / upper() / title() / capitalize() / swapcase()	Case conversion
s.strip() / lstrip() /rstrip(chars)	Trim whitespace or given chars
s.split(sep=None, maxsplit=-1)	String to list (no sep = split on any whitespace)
sep.join(iterable)	List of str to one str
s.replace(old, new, count)	Replace occurrences (new string)
s.find(x) / rfind(x)	Index of first/last match, -1 if none
s.index(x) / rindex(x)	Like find but raises ValueError
s.count(x)	Non-overlapping occurrences
s.startswith(x) / endswith(x)	Prefix / suffix test (tuple allowed)
s.isdigit() / isalpha() / isalnum() / isspace()	Character class tests
s.islower() / isupper()	Case tests
s.zfill(n) / center(n) / ljust(n) / rjust(n)	Padding
s.partition(sep) / rpartition(sep)	(before, sep, after) tuple
s.splitlines()	Split on line breaks
s.format(...) / f-strings	Formatting
s.encode() / bytes.decode()	str <-> bytes
s.translate(str.maketrans(a, b))	Character mapping / deletion

7.4 List Methods Cheat Sheet

Method	Effect	Time
a.append(x)	Add x at end	O(1)
a.extend(it)	Add each item of it	O(k)
a.insert(i, x)	Insert before index i	O(n)
a.remove(x)	Delete first x (ValueError if none)	O(n)
a.pop() / a.pop(i)	Remove and return (last / index i)	O(1) / O(n)
del a[i] / del a[i:j]	Delete by index / slice	O(n)
a.clear()	Empty the list	O(n)
a.index(x)	Index of first x	O(n)
a.count(x)	Occurrences of x	O(n)
a.sort(key=, reverse=)	In-place sort, returns None	O(n log n)

Method	Effect	Time
<code>a.reverse()</code>	In-place reverse, returns None	$O(n)$
<code>a.copy()</code> / <code>a[:]</code>	Shallow copy	$O(n)$
<code>x in a</code>	Membership test	$O(n)$
<code>a[i]</code> / <code>a[i] = v</code>	Index read / write	$O(1)$
<code>a + b</code> / <code>a * k</code>	New list (concat / repeat)	$O(n)$

7.5 Set Methods Cheat Sheet

Operation	Meaning	Time (avg)
<code>s.add(x)</code>	Add element	$O(1)$
<code>s.remove(x)</code>	Delete; <code>KeyError</code> if missing	$O(1)$
<code>s.discard(x)</code>	Delete; silent if missing	$O(1)$
<code>s.pop()</code>	Remove and return an arbitrary element	$O(1)$
<code>s.clear()</code>	Empty the set	$O(n)$
<code>s.update(it)</code> / <code>s = t</code>	Add many	$O(k)$
<code>A B</code> / <code>A.union(B)</code>	Union	$O(n+m)$
<code>A & B</code> / <code>A.intersection(B)</code>	Intersection	$O(\min)$
<code>A - B</code> / <code>A.difference(B)</code>	In A but not B	$O(n)$
<code>A ^ B</code> / <code>A.symmetric_difference(B)</code>	In exactly one	$O(n+m)$
<code>A <= B</code> / <code>A.issubset(B)</code>	Subset test	$O(n)$
<code>A >= B</code> / <code>A.issuperset(B)</code>	Superset test	$O(m)$
<code>A.isdisjoint(B)</code>	No common items	$O(\min)$
<code>x in s</code>	Membership	$O(1)$
<code>frozenset(it)</code>	Immutable, hashable set	$O(n)$

7.6 Dictionary Methods Cheat Sheet

Method	Effect	Time (avg)
<code>d[k]</code> / <code>d[k] = v</code>	Read (<code>KeyError</code> if missing) / write	$O(1)$
<code>d.get(k, default)</code>	Safe read, no error	$O(1)$
<code>d.setdefault(k, default)</code>	Get, inserting default if missing	$O(1)$
<code>d.pop(k, default)</code>	Remove and return value	$O(1)$
<code>d.popitem()</code>	Remove and return last inserted pair	$O(1)$
<code>del d[k]</code>	Delete key	$O(1)$
<code>d.update(other)</code> / <code>d = other</code>	Merge (other wins)	$O(m)$
<code>d.keys()</code> / <code>values()</code> / <code>items()</code>	Live views	$O(1)$ to make
<code>k in d</code>	Key membership	$O(1)$
<code>d.copy()</code>	Shallow copy	$O(n)$
<code>dict.fromkeys(keys, v)</code>	New dict with same value	$O(n)$
<code>{k: v for k, v in it}</code>	Dict comprehension	$O(n)$
<code>sorted(d.items(), key=lambda p: p[1])</code>	Sort by value	$O(n \log n)$

Method	Effect	Time (avg)
d1 d2	New merged dict (3.9+)	O(n+m)

7.7 Collections Module Cheat Sheet

Tool	Use it for	Key calls
Counter	Frequency counting	Counter(a); c.most_common(k); c[x]; c1 + c2; c1 - c2
defaultdict	Auto-created values	defaultdict(list); defaultdict(int); d[k].append(v)
deque	O(1) at both ends (queue, sliding window)	append; appendleft; pop; popleft; rotate(k); deque(maxlen=k)
OrderedDict	Order control, LRU	move_to_end(k); popitem(last=False)
namedtuple	Lightweight record	P = namedtuple('P','x y'); p = P(1,2); p.x
ChainMap	Layered lookups	ChainMap(local, defaults)
heapq	Min-heap priority queue	heapify(a); heappush(h,x); heappop(h); nlargest(k,a); nsmallest(k,a); heappushpop
bisect	Binary search on sorted lists	bisect_left(a,x); bisect_right(a,x); insort(a,x)
itertools	Iterator building blocks	permutations; combinations; product; accumulate; chain; groupby; islice; count; cycle; zip_longest
functools	Function tools	lru_cache / cache; reduce; partial; wraps; cmp_to_key; total_ordering
math	Numbers	sqrt; gcd; lcm; ceil; floor; factorial; comb; perm; inf; isqrt
copy	Copying	copy.copy(x); copy.deepcopy(x)

7.8 DSA Templates

Copy-ready skeletons for the patterns that solve most interview problems.

Two pointers (sorted array / pair)

```
l, r = 0, len(a) - 1
while l < r:
    s = a[l] + a[r]
    if s == target: return [l, r]
    if s < target: l += 1
    else: r -= 1
```

Sliding window (variable size)

```
left = 0
for right, x in enumerate(a):
    add(x) # include a[right]
    while window_is_invalid():
        remove(a[left]); left += 1
    best = max(best, right - left + 1)
```

Prefix sum

```
pref = [0]
for x in a: pref.append(pref[-1] + x)
range_sum = pref[j + 1] - pref[i] # sum of a[i..j]
```

Binary search (first index where cond is True)

```
l, r = 0, len(a)
while l < r:
    m = (l + r) // 2
    if cond(a[m]): r = m
    else: l = m + 1
return l
```

Frequency map / set for seen

```
from collections import Counter
freq = Counter(a)
seen = set()
for x in a:
    if x in seen: ...
    seen.add(x)
```

Monotonic stack (next greater)

```
st = [] # indexes
for i, x in enumerate(a):
    while st and a[st[-1]] < x:
        j = st.pop(); ans[j] = i - j
    st.append(i)
```

BFS (shortest path. unweighted)

```
from collections import deque
q = deque([start]); seen = {start}; dist = {start: 0}
while q:
    u = q.popleft()
    for v in adj[u]:
        if v not in seen:
            seen.add(v); dist[v] = dist[u] + 1; q.append(v)
```

DFS (recursive and iterative)

```
def dfs(u):
    seen.add(u)
    for v in adj[u]:
        if v not in seen: dfs(v)
# iterative
st = [start]
while st:
    u = st.pop()
    if u in seen: continue
    seen.add(u); st.extend(adj[u])
```

Backtracking

```
def go(start, path):
    if is_solution(path): res.append(path[:]); return
    for i in range(start, n):
        path.append(a[i])
        go(i + 1, path) # i for reuse
        path.pop() # undo choice
```

Heap top-k / Diikstra

```
import heapq
h = []; heapq.heappush(h, (dist, node))
while h:
    d, u = heapq.heappop(h)
    if d > best.get(u, inf): continue
    for v, w in adj[u]: ...
```

1D dynamic programming

```
dp = [base] + [0] * n
for i in range(1, n + 1):
    dp[i] = combine(dp[i - 1], dp[i - 2], a[i - 1])
return dp[n]
```

Union-Find

```
parent = list(range(n))
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]; x = parent[x]
    return x
def union(a, b): parent[find(a)] = find(b)
```

7.9 Big-O Cheat Sheet

Data structure operations

Structure	Access	Search / in	Insert	Delete
list	$O(1)$	$O(n)$	append $O(1)^*$, insert $O(n)$	pop() $O(1)$, pop(i)/remove $O(n)$
tuple	$O(1)$	$O(n)$	immutable	immutable
dict	$O(1)$ by key	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
set	n/a	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg
deque	$O(n)$ middle	$O(n)$	$O(1)$ both ends	$O(1)$ both ends
heap (heapq)	$O(1)$ min	$O(n)$	push $O(\log n)$	pop $O(\log n)$
sorted list + bisect	$O(1)$	$O(\log n)$	insort $O(n)$	$O(n)$
string	$O(1)$	find $O(n)$	concat $O(n)$	immutable

Common algorithms / patterns

Algorithm / pattern	Time	Space
Linear scan, hash-map single pass	$O(n)$	$O(n)$
Binary search	$O(\log n)$	$O(1)$
sort() / sorted() (Timsort)	$O(n \log n)$	$O(n)$
Two pointers / sliding window	$O(n)$	$O(1)$ to $O(k)$
Heap top-k	$O(n \log k)$	$O(k)$
Nested loops over one array	$O(n^2)$	$O(1)$
Subsets / permutations	$O(2^n) / O(n!)$	$O(n)$ recursion
BFS / DFS on graph	$O(V + E)$	$O(V)$
Dijkstra with heap	$O((V+E) \log V)$	$O(V)$
2D DP ($n \times m$)	$O(n*m)$	$O(n*m)$, often reducible
Slicing $a[i:j]$	$O(j-i)$	$O(j-i)$
Recursion depth limit	~1000 frames	sys.setrecursionlimit

* amortized. Order of growth: $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n) < O(n!)$. With $n = 10^5$ aim for $O(n \log n)$ or better; n^2 is about 10^{10} and too slow.

7.10 Python Interview Traps

Trap	The bug	The fix
Mutable default argument	<code>def f(x, L=[])</code>	<code>def f(x, L=None): L = [] if L is None else L</code>
Aliasing instead of copying	<code>b = a</code>	<code>b = a.copy() / copy.deepcopy(a)</code>
Repeated inner list	<code>[[0]*3]*3</code>	<code>[[0]*3 for _ in range(3)]</code>
Using the return of in-place methods	<code>a = a.sort()</code> (None)	<code>a.sort()</code> or <code>a = sorted(a)</code>
Mutating while iterating	<code>for x in a: a.remove(x)</code>	<code>for x in a[:]:</code> or <code>a = [x for x in a if keep(x)]</code>
is for values	<code>if x is 5</code>	<code>if x == 5</code> (is only for None/True/False)
Late-binding lambdas	<code>[lambda: i for i in range(3)]</code>	<code>[lambda i=i: i for i in range(3)]</code>
UnboundLocalError	<code>def f(): x += 1 (global x)</code>	<code>global x / nonlocal x / pass as argument</code>
Empty dict vs set	<code>s = {}</code>	<code>s = set()</code>
Tuple needs a comma	<code>t = (5)</code>	<code>t = (5,)</code>
Integer vs float division	<code>7 / 2</code> gives 3.5	<code>7 // 2</code> gives 3
Negative floor division / mod	<code>-7 // 2 = -4, -7 % 3 = 2</code>	remember floor toward -infinity
Bare except	<code>except:</code>	<code>except (ValueError, KeyError) as e:</code>
Exhausted iterators	<code>g = (...); list(g); list(g) -> []</code>	recreate the generator or store a list
list.pop(0) in a loop	O(n) each	<code>collections.deque.popleft()</code>
String concatenation in loops	<code>s += piece</code>	<code>parts.append(piece); ''.join(parts)</code>
Sorting stability / key	sort by two keys	<code>key=lambda p: (-p[1], p[0])</code>
Shadowing built-ins	<code>list = [1,2]; sum = 0</code>	use names like <code>lst</code> , <code>total</code>
Float equality	<code>0.1 + 0.2 == 0.3</code> (False)	<code>math.isclose(a, b)</code>
Class vs instance attribute	<code>items = []</code> at class level	<code>self.items = []</code> in <code>__init__</code>
Missing key	<code>d[k]</code> KeyError	<code>d.get(k, 0)</code> / <code>defaultdict</code> / <code>Counter</code>
Off-by-one in slices/range	<code>range(1, n)</code> misses <code>n</code>	<code>range(1, n + 1)</code> ; stop is exclusive

Trap	The bug	The fix
Recursion without a base case	RecursionError	always define base case and shrink the input
Returning vs printing	<code>print(x)</code> inside function	return x (print only at the end)
<code>==</code> on hashable custom class	<code>__eq__</code> without <code>__hash__</code>	define both, or use <code>dataclass(frozen=True)</code>

7.11 Most Important Terminology (Quick List)

Term	One-line meaning
mutable / immutable	changes in place / never changes
iterable / iterator	can be looped / yields via next()
generator	function with yield; lazy, one pass
hashable	stable hash; usable as dict key / set item
reference	a name pointing to an object
identity vs equality	is vs ==
shallow / deep copy	shares inner objects / duplicates all
closure	inner function remembering outer variables
decorator	function that wraps a function
LEGB	Local, Enclosing, Global, Built-in
MRO	class lookup order (C3), D.__mro__
duck typing	judged by behaviour, not class
dynamic + strong typing	types checked at runtime, no silent coercion
context manager	with-block setup and cleanup
comprehension	compact list/set/dict builder
lambda	one-expression anonymous function
polymorphism	same call, different behaviour
encapsulation	data + methods, controlled access
abstraction	hide how, show what
inheritance	reuse and extend a class
namespace / scope	name mapping / where a name is visible
module / package	one file / a folder of modules
recursion	function calls itself; base case needed
hash table	key hashed to a slot, avg O(1)

7.12 One-Page Final Revision Sheet

The whole book compressed. Read this page the morning of the interview.

Core types

Mutable: list dict set | Immutable: int float str tuple bool frozenset
Hashable (dict key): immutables of hashables | empty: {} is dict, set() is set
Falsy: False None 0 0.0 "" [] {} set() ()

Slicing

a[start:stop:step] stop excluded | a[::-1] reverse | a[:] copy | a[-1] last
Slice makes a NEW list | slice assign may resize (extended slice may not)

Copy / identity

b = a same object | a.copy() a[:] list(a) shallow | copy.deepcopy(a) deep
== value, is identity | [[0]*3]*3 shares rows -> use a comprehension

Functions

default args evaluated ONCE -> use None | *args tuple, **kwargs dict
no return -> None | local if assigned anywhere in function | global / nonlocal
LEGB lookup | closures capture variables (late binding)

List methods

append 1 | extend many | insert(i,x) | remove(x) by value | pop(i) by index+returns
sort() in place->None | sorted() new list | reverse() | index count copy clear

Dict / set / Counter

d.get(k,0) |.setdefault | items() | defaultdict(list) | Counter(a).most_common(k)
set: add discard remove | | & - ^ | x in set is O(1) | dict/set preserve nothing for sets

Iteration

enumerate(a,1) | zip(a,b) | reversed | sorted(key=) | map filter any all
iterable -> iter() -> iterator -> next() -> StopIteration | generators exhaust after 1 pass

Exceptions

try / except (specific) / else (no error) / finally (always) | raise | raise X from e
finally runs even after return | never bare except

OOP

__init__ self | class vs instance attrs | @property @staticmethod @classmethod
__str__ __repr__ __eq__ __lt__ __len__ __add__ | super() follows MRO | ABC + @abstractmethod

Complexity

list append O(1), insert/pop(0)/remove/in O(n) | dict, set O(1) avg | deque ends O(1)
sort O(n log n) | heap push/pop O(log n) | binary search O(log n) | slice O(k)

Patterns

hash map (Two Sum) | two pointers (sorted) | sliding window (substring, subarray)
prefix sum + map (subarray sum k) | binary search | monotonic stack | heap top-k
BFS shortest path | DFS / backtracking | DP (table of subproblems)

Interview habits

clarify input, edge cases (empty, one item, duplicates, negatives) | brute force first
state complexity | test with a small example | name variables clearly | talk while coding

SECTION 8 — Final Mastery Checklist

The progression to hold yourself to, stage by stage. Do not rush a stage — each one is the foundation for the next.

1. Beginner Python

- Variables, references, object identity (id, type)
- Core types: int, float, bool, str, None and their conversions
- Operators and operator precedence
- input()/output(), especially map(int, input().split())
- if / elif / else and truthiness
- for and while loops, break / continue / pass
- Basic functions: parameters, return vs print



2. Strong Python Fundamentals

- Strings: slicing, immutability, all core methods
- Lists: every method, slicing, time complexity of each op
- Tuples: unpacking, why they exist, hashability
- Sets: operations, why membership is O(1)
- Dictionaries: get/setdefault, frequency counting pattern
- Mutable vs immutable, is vs ==, shallow vs deep copy



3. Pythonic Programming

- List / set / dict comprehensions
- enumerate, zip, map, filter, any, all
- Lambda functions and sort keys
- *args, **kwargs, unpacking with *
- Ternary operator, walrus operator
- Exception handling: try/except/else/finally



4. Advanced Python

- Iterables vs iterators vs generators, yield, lazy evaluation
- Closures, decorators, context managers

- OOP: classes, dunder methods, inheritance, MRO
- Scope and LEGB, global/nonlocal
- Modules, packages, standard library tour



5. DSA with Python

- collections: Counter, defaultdict, deque, namedtuple
- heapq and bisect for heaps and binary search
- Two pointers, sliding window, prefix sums
- Stacks, queues, monotonic stacks
- Recursion, backtracking, BFS/DFS
- 1D and 2D dynamic programming



6. Interview Preparation

- Predict-the-output questions until instinctive
- Solve beginner -> intermediate -> advanced sets in this book
- Practice explaining approach and complexity out loud
- Time yourself; practice on a blank editor, no autocomplete
- Review the interview traps sheet the day before



7. AI/ML Python

- NumPy: arrays, broadcasting, vectorized operations
- Pandas: DataFrames, filtering, groupby, merge
- Comfort reading and writing vectorized code instead of loops
- Matplotlib/Seaborn basics for quick visualization
- Reading ML library docs comfortably (sklearn, PyTorch/TensorFlow basics)

End of Part 7. This workbook, together with Parts 1-6, covers Python from first principles through interview-ready fluency.